

PROJECT SYNOPSIS

Video Streaming Platform

A Secure Video Streaming Platform with Adaptive Quality and Management Tools

STATEMENT OF THE PROBLEM

Online video streaming is very popular for education and entertainment. However, many current platforms are expensive, show too many ads, or have limited tools for new creators. Additionally, viewers often experience buffering and poor video quality when their internet connection is slow.

The proposed Video Streaming Platform solves these issues. It provides an easy-to-use website where users can register, upload videos, and stream content smoothly. It also includes adaptive streaming, which automatically changes the video quality based on the viewer's network conditions to reduce buffering.

PROBLEM DESCRIPTION

The proposed system is a modern video streaming website built to be better than simple filesharing platforms. It gives users and creators a clean and responsive interface that works well on both computers and mobile phones. To handle large video files properly without slowing down the server, the project uses Cloudflare R2 is used to store processed video files while HLS is used to stream videos in multiple quality levels.

The system uses a tool called FFmpeg for video processing. When a registered creator uploads a video, FFmpeg automatically converts it into multiple qualities (360p, 720p, and 1080p). The platform utilizes HTTP Live Streaming (HLS) to serve these varying resolutions, enabling true adaptive streaming. To manage storage effectively, the system enforces creator upload limits, maximum video file sizes, allowed video formats, and the deletion of unused files.

The project prioritizes core functionalities like secure login, video processing, streaming, search, and comments. As additional interactive features, the platform uses Socket.io (WebSocket's) for synchronized "Watch Parties" with live chat, and the Stripe API will be used in test mode for demonstrating subscriptions payments. The system uses modular layout, building major parts like User Profiles, Video Uploads, and Media Management as separate modules. This keeps the code clean, easy to test, and easy to upgrade.

OBJECTIVE AND SCOPE OF THE PROJECT

Objectives

- Build a secure and fast video streaming website using the MERN stack.
- Create a safe registration and login system for users.
- Allow creators to upload, edit, and manage their own videos.
- Add an adaptive video player that switches quality based on internet speed.
- Design a fully responsive user interface that looks great on mobile screens.

Scope

This project includes a complete website for uploading, browsing, searching, and watching videos. The main features are secure user accounts, automated video quality compression, real-time synchronized watch parties with live chat, and subscription billing using Stripe. Advanced features like automated AI subtitles, a mobile app, and a personalized recommendation algorithm are planned as future updates.

METHODOLOGY

The project will be built step by step using the following stages:

- Requirement Analysis & Planning: Study project features, identify user needs, and create a basic project plan.
- System Design: Plan the database structure, website layout, and draw the system diagrams.
- Frontend Development: Create the pages (Home, Login, Upload, Video Player, Search) using HTML5, CSS3, JavaScript, and React.js.
- Backend Development: Build the server logic, user authentication, and upload controls using Node.js and Express.js.
- Database Integration: Connect MongoDB to store user data, video information, comments, and channels.
- Video Processing Setup: Configure FFmpeg on the server to automatically transcode videos into multiple streaming resolutions.
- Real-time Watch Parties: Use Socket.io to synchronize play, pause, and skip actions across users in the same viewing room.

- Payment Integration: Stripe test mode
- Testing & Deployment: Test all features together, fix bugs, and deploy the application for the final presentation.

HARDWARE & SOFTWARE

Category	Details
Hardware	Intel Core i3 or higher processor, 8 GB RAM or more, 256 GB SSD, stable internet connection, development laptop or desktop
Frontend	HTML, CSS, JavaScript, React.js (using video.js or HLS.js libraries)
Backend	Node.js, Express.js frameworks
Database	MongoDB (NoSQL database)
Video Processing	FFmpeg transcodes videos into multiple resolutions and generates HLS streaming files/playlists.
Streaming & Storage	Cloudflare R2 is for storing processed video files and HLS streaming content
Real-time Actions	Socket.io (for live chat and synchronized video playback controls)
Payments	Stripe Subscriptions API (TEST MODE)
Developer Tools	Visual Studio Code editor, Git for version tracking, Postman for API testing, Google Chrome

PROCESS DESCRIPTION & SYSTEM ACTORS

The system has two main types of users:

- Viewer: Can log in, search or browse for videos, join real-time watch parties with friends, send chat messages, and purchase monthly subscriptions.
- Creator: A registered creator account that can manage their channel profile, view video details, and upload new video files subject to specific size, format, and quota limits.

RESOURCES AND LIMITATIONS

Resources

- **Hardware Tools:** A standard computer with an Intel i3 processor, 8 GB RAM, and a standard internet connection.
- **Software Tools:** Windows OS, Visual Studio Code text editor, and Google Chrome web browser.
- **People:** The student developer, the college project mentor, and target test users.

Limitations

- The website requires an internet connection for video streaming; unstable network conditions may affect playback quality and cause buffering.
- Large video files take up a lot of storage space and need more processing time to convert.
- Video upload speeds are entirely limited by the user's home internet connection
- Because development is done on an i3 laptop, video transcoding performance may be limited by the development hardware, especially for large or high-resolution videos.

FUTURE WORK

- An AI-based video recommendation system to suggest content based on watch history.
- Dedicated mobile applications for Android and iOS devices.
- An analytics dashboard so creators can see how many views and watch hours they have.
- An offline download feature using Progressive Web App (PWA) tools.
- Automated closed captions using OpenAI's Whisper speech-to-text model to generate subtitles.

PROJECT SCHEDULE (GANTT CHART)

Task	Week 1	Week 2	Week 3	Week 4
Requirement Analysis & Planning				
System Design & Architecture				
Frontend Development (UI & Video Player)				
Backend Development & APIs				
Database & Cloud Storage Integration				
Testing, WebSocket's & Security Debugging				
Documentation & Code Refactoring				
Final Deployment & Demonstration				

CONCLUSION

The Video Streaming Platform is a web-based application designed to provide a simple, secure, and user-friendly way for users to upload, manage, and stream videos online. The platform supports smooth video playback with adaptive streaming, ensuring a better viewing experience even with different internet speeds. It also offers secure user authentication and organized video management for content creators. Overall, this project demonstrates how modern web technologies can be used to build an efficient and reliable video streaming system that benefits both creators and viewers. In the future, additional features such as live streaming, personalized video recommendations, and mobile application support can be added to further expand the platform's capabilities.

REFERENCES

1. Node.js Runtime Environment, San Francisco, CA: openJS Foundation, 2026. Available: <https://nodejs.org>
2. Opens Foundation, Express.js Application Framework: <https://nodejs.org>
3. Meta Open Source, React.js Frontend Library Documentation. Available: <https://react.dev>
4. MongoDB Inc., MongoDB Database Architecture Manual. Available: <https://www.mongodb.com/docs/>
5. FFmpeg Project, FFmpeg Multimedia Framework Documentation: <https://ffmpeg.org/documentation.html>
6. Cloudflare Inc., Cloudflare R2 Distributed Storage Architecture: <https://developers.cloudflare.com/r2/>
7. Socket.io, Socket.io Real-Time Engine Documentation: <https://socket.io/docs/v4>